

Assignment 1: QuickShop — E-Commerce Catalog API

Pending



Objective

Build a **FastAPI-based E-Commerce Catalog API** where users can browse products and admins can manage products and stock.

The assignment should demonstrate:

- FastAPI
- SQLAlchemy
- JWT Authentication
- Role-Based Access Control
- Pydantic Validation
- Enum
- Route Grouping

Technical Requirements

- **Framework:** FastAPI
- **ORM:** SQLAlchemy / Async SQLAlchemy
- **Database:** PostgreSQL or SQLite
- **Password Hashing:** bcrypt or passlib
- **Authentication:** JWT using OAuth2PasswordBearer
- **Roles:** admin, user

Database Models

User

- id
- email
- password_hash
- role

Product

- id
 - name
 - description
 - price
 - stock
 - category
-

Product Categories

Use an Enum with the following values:

- electronics
 - fashion
 - grocery
 - books
 - home
-

API Routes

POST /auth/register

Register a new user.

Requirements:

- Email must be unique.

- Password must be hashed before storing.

POST /auth/token

Login using valid credentials and return a **JWT access token**.

GET /products

List available products.

Support optional filters:

- min_price
- max_price
- category

Example:

`/products?min_price=500&max_price=5000&category=electronics`

POST /products

Admin Only

Add a new product.

Validation:

- price > 0
 - stock >= 0
 - Category must be a valid Enum value.
-

PUT /products/{id}

Admin Only

Update an existing product using a Pydantic update schema.

Validate the updated values before saving them.

Mandatory Validation

- `price > 0`
- `stock >= 0`
- Valid product category
- Unique email
- Password must be hashed
- Only admins can create or update products

Use Pydantic `Field()` wherever applicable.

Example:

```
price: float = Field(gt=0)
stock: int = Field(ge=0)
```

Suggested Pydantic Schemas

- `UserCreate`
 - `UserResponse`
 - `Token`
 - `ProductCreate`
 - `ProductUpdate`
 - `ProductResponse`
-

Route Grouping

Use FastAPI `APIRouter` to organize routes.

```
/auth
    /register
    /token
```

```
/products
    GET /
    POST /
    PUT /
```

Error Handling

Handle appropriate errors for:

- Duplicate email
- Invalid login
- Invalid or expired JWT
- Unauthorized access
- Product not found
- Invalid category
- Invalid price or stock

Use proper HTTP status codes such as 400, 401, 403, 404, and 422.

API Testing

All APIs should be testable through FastAPI Swagger:

/docs

Use the **Authorize** option to test JWT-protected APIs.

Submission Instructions

Submit the **complete project as ONE ZIP file through the LMS.**

The ZIP file should include:

- Complete source code
- requirements.txt
- README.md
- All files required to run the project

ZIP File Name

StudentName_QuickShop.zip

Example:

Raju_QuickShop.zip

Important

- Submit only **one ZIP file**.
 - Do not upload individual files separately.
 - Do not include `venv`, `.venv`, `__pycache__`, or other unnecessary folders.
 - Test the application before submission.
 - Ensure the correct ZIP file is uploaded successfully to the LMS.
 - Submit before the deadline mentioned in the LMS.
-

Evaluation Criteria

The assignment will be evaluated based on:

- Correct API functionality
- JWT Authentication
- Admin/User role implementation
- Pydantic and Enum validation
- SQLAlchemy database implementation
- Product filtering
- Error handling
- Proper route organization
- Code quality
- Successful project execution
- Proper ZIP submission

Expected Outcome

Expected Outcome

By completing this assignment, you should be able to build a structured **FastAPI backend with database integration, JWT authentication, role-based access, validation, filtering, and protected APIs.**

Upload Assignment